

Archivos

* Definición

Un **archivo** es una unidad básica de almacenamiento de información en un sistema de archivos. Contiene datos que pueden ser texto, imágenes, audio, videos, programas o cualquier tipo de información digital.

Características clave de un archivo:

1. **Nombre**: Cada archivo tiene un nombre único dentro de un directorio.
2. **Extensión**: El sufijo que indica el tipo de archivo (por ejemplo, `.txt`, `.jpg`, `.exe`), aunque no es obligatorio en todos los sistemas operativos.
3. **Metadatos**: Atributos del archivo, como tamaño, fecha de creación/modificación, permisos de acceso, y en algunos sistemas, su ubicación exacta en el disco.
4. **Dirección**: El archivo tiene una ubicación dentro del sistema de archivos, definida por la ruta (ej., `/home/user/document.txt`).

Los archivos se gestionan mediante operaciones como crear, abrir, leer, escribir, y eliminar.

La información que se almacena en los archivos debe ser **persistente**, es decir, no debe ser afectada por la creación y terminación de los procesos.

Operaciones de archivos

Llamadas al sistema más comunes relacionadas con los archivos:

- `Create` → el archivo se crea sin datos. Se establecen algunos atributos
- `Delete` → se elimina para liberar espacio en el disco o porque no se necesita
- `Open` → antes de usar un archivo, se debe abrir. El sistema debe llevar los atributos y listas de direcciones de disco a memoria principal para tener un acceso rápido
- `Close` → el archivo debe cerrarse para liberar espacio en la tálpa interna
- `Read` → los datos se leen del archivo
- `Write` → los datos se escriben en el archivo
- `Append` → función restringida de write, agrega datos al final del archivo
- `Seek` → método para acceso aleatorio al archivo que reposiciona el apuntador del archivo en una posición específica del archivo
- `Get attributes` → leer atributos de un archivo
- `Set attributes` → establecer atributos de un archivo creado
- `Rename` → cambiar el nombre de un archivo
- `Link`
- `Unlink`

Comparación entre Win32 y POSIX:

Aspecto	Win32 (Windows)	POSIX (Unix/Linux)
Longitud máxima del archivo	260 caracteres (por defecto)	255 caracteres por archivo (4096 caracteres para la ruta completa)
Sensibilidad a mayúsculas	No es sensible (ej. <code>archivo.txt</code> = <code>ARCHIVO.txt</code>)	Sensible (ej. <code>archivo.txt</code> ≠ <code>ARCHIVO.txt</code>)
Caracteres prohibidos	<code>\</code> , <code>/</code> , <code>:</code> , <code>*</code> , <code>?</code> , <code>"</code> , <code><</code> , <code>></code> , <code>`</code>	Solo <code>/</code> y <code>NULL</code> , nombres reservados (<code>CON</code> , <code>PRN</code> , etc.)
Extensiones	Se usan comúnmente, aunque no obligatorias	No son necesarias para el sistema, usadas por convención
Separador de directorios	<code>\</code> (barra invertida)	<code>/</code> (barra)

Aspecto	Win32 (Windows)	POSIX (Unix/Linux)
Ruta de archivo	Incluye letra de unidad (ej. <code>C:\</code>)	Relativa a la raíz del sistema (ej. <code>/home/</code>)

Ejemplo completo

Si ejecutas el comando `ls -l` en un archivo con estos permisos, verás algo como esto:

```
-rwxrw-r-- 1 usuario grupo tamaño fecha nombre_de_archivo
```

- **-rwxrw-r--**: Los permisos 🗑️.
- **1**: Número de enlaces duros (links).
- **usuario**: El dueño del archivo.
- **grupo**: El grupo asociado al archivo.
- **tamaño**: Tamaño del archivo en bytes.
- **fecha**: Fecha y hora de la última modificación.
- **nombre_de_archivo**: Nombre del archivo.
- El primer carácter (`-`) indica si es un archivo regular
- **Permisos del usuario (dueño)**:
 - **r**: El usuario puede leer el archivo.
 - **w**: El usuario puede escribir (modificar) el archivo.
 - **x**: El usuario puede ejecutar el archivo (si es un archivo ejecutable).
- **Permisos del grupo**:
 - **r**: Los miembros del grupo pueden leer el archivo.
 - **w**: Los miembros del grupo pueden escribir (modificar) el archivo.
 - **-**: Los miembros del grupo no tienen permiso para ejecutar el archivo.
- **Permisos para otros (cualquiera)**:
 - **r**: Cualquier otro usuario puede leer el archivo.
 - **-**: Cualquier otro usuario no puede escribir ni modificar el archivo.
 - **-**: Cualquier otro usuario no puede ejecutar el archivo.

En sistemas Linux/Unix, el primer carácter en la salida del comando `ls -l` indica el **tipo de archivo o elemento** que estamos viendo. Aquí te dejo una lista con los diferentes tipos de archivos o elementos y sus correspondientes caracteres:

Tipos de archivos y su representación:

Carácter	Tipo de archivo o elemento	Descripción
-	Archivo ordinario	Un archivo normal, como un documento de texto, imagen, etc.
d	Directorio	Un directorio o carpeta que contiene otros archivos o directorios.
b	Block special file	Archivo de dispositivo de bloque, usado para interactuar con hardware que maneja bloques de datos, como discos duros.
c	Character special file	Archivo de dispositivo de caracteres, usado para interactuar con dispositivos que manejan flujos de datos carácter por carácter (ej. impresoras, terminales, teclados, redes).
p	Named pipe (FIFO)	Canal de comunicación de interprocesos con nombre. Facilita la transferencia de datos entre procesos.
l	Link simbólico	Un enlace simbólico (o "symlink") que apunta a otro archivo o directorio.

Posibles atributos para un archivos

Attribute	Meaning
Protection	Who can access the file and in what way
Password	Password needed to access the file
Creator	ID of the person who created the file
Owner	Current owner
Read-only flag	0 for read/write; 1 for read only
Hidden flag	0 for normal; 1 for do not display in listings
System flag	0 for normal files; 1 for system file
Archive flag	0 for has been backed up; 1 for needs to be backed up
ASCII/binary flag	0 for ASCII file; 1 for binary file
Random access flag	0 for sequential access only; 1 for random access
Temporary flag	0 for normal; 1 for delete file on process exit
Lock flags	0 for unlocked; nonzero for locked
Record length	Number of bytes in a record
Key position	Offset of the key within each record
Key length	Number of bytes in the key field
Creation time	Date and time the file was created
Time of last access	Date and time the file was last accessed
Time of last change	Date and time the file has last changed
Current size	Number of bytes in the file
Maximum size	Number of bytes the file may grow to

Mapeo en memoria

El **mapeo de memoria** es una técnica que permite asociar archivos directamente con el espacio de direcciones del proceso, de modo que el archivo pueda ser accedido como si fuera parte de la memoria principal, sin necesidad de operaciones de E/S tradicionales (lectura/escritura).

1. Proporcionan una forma de asociar archivos con el espacio de direcciones del proceso:

- Un archivo en disco se mapea directamente a una porción de la memoria virtual del proceso.
- Esto permite acceder a los contenidos del archivo como si fueran parte de la memoria, sin tener que leer/escribir manualmente desde/hacia el disco.

2. Ventaja: Minimiza E/S:

- Al mapear el archivo en memoria, se reduce el número de operaciones de E/S, lo que puede mejorar el rendimiento. Esto es útil cuando se requiere acceso frecuente y rápido a datos almacenados en archivos.

3. Desventajas: Solo para pequeños archivos:

- Esta técnica es más eficiente para archivos pequeños. Si un archivo es muy grande, puede ser difícil o imposible mapear todo el archivo en la memoria disponible, lo que limita su uso en esos casos.

4. Inconsistencia si dos procesos acceden al archivo de manera diferente:

- Si un proceso mapea un archivo a su memoria y otro proceso accede al archivo de forma convencional (por ejemplo, a través de operaciones de E/S estándar), pueden producirse **inconsistencias**. Esto se debe a que el proceso que accede de forma convencional no verá automáticamente los cambios que se hagan a través del mapeo, y viceversa.

5. ¿Qué pasa si el archivo ocupa más de un segmento?:

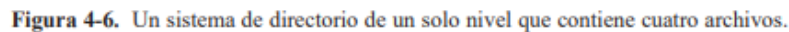
- En sistemas de memoria segmentada, si el archivo es más grande que el tamaño de un segmento, se divide en múltiples segmentos. El sistema operativo se encarga de gestionar la asignación y paginación de esos segmentos en la memoria virtual del proceso.
- Cuando se accede a una parte del archivo que no está en memoria, el sistema operativo intercambia los segmentos de memoria necesarios, lo que podría degradar el rendimiento si el archivo es muy grande.

Directorios

Son también archivos que llevan el registro de archivos.

1. Directorios de un nivel:

- **Estructura:** Todos los archivos se almacenan en un único directorio común.
- **Ventaja:** Simplicidad en la administración.
- **Desventaja:** No permite la organización eficiente cuando hay muchos archivos, ya que todos están en el mismo nivel y pueden tener conflictos de nombres.



Operaciones con directorios

- 📁 **Create** → se crea un directorio
- 📁 **Delete** → se elimina un directorio
- 📁 **Opendir** → antes de poder leer un directorio, hay que abrirlo
- 📁 **Closedir** → se debe cerrar para liberar espacio en la tabla interna
- 📁 **Readdir** → devuelve la siguiente entrada en un directorio abierto
- 📁 **Rename** → se puede cambiarse el nombre
- 📁 **Link** → enlace físico o simbólico
- 📁 **Unlink** → eliminar de la tabla el link

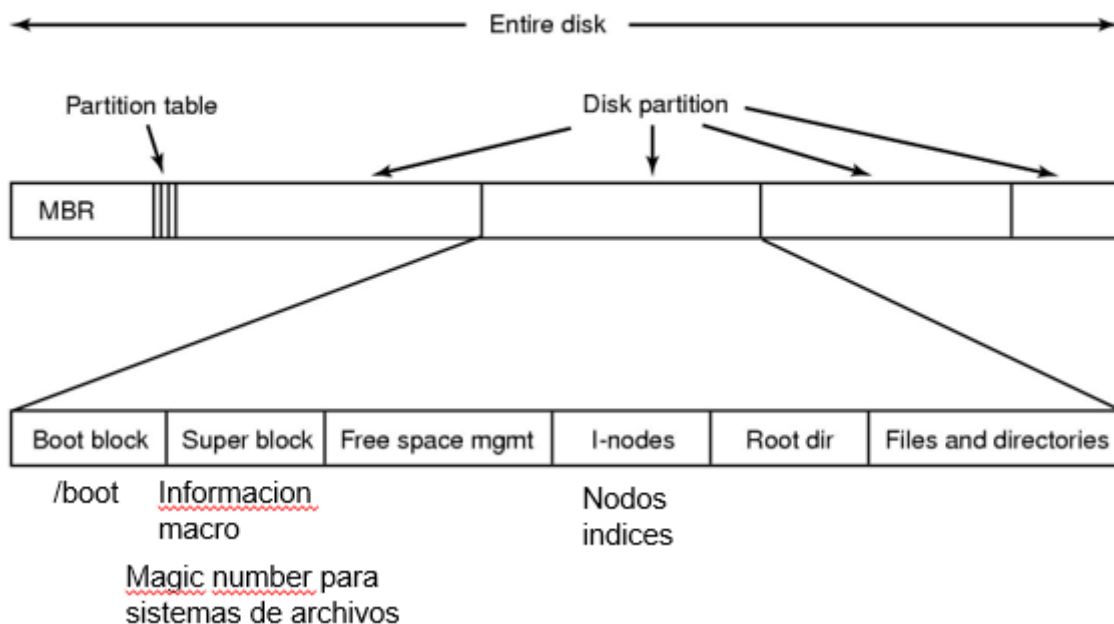
Tipo de Enlace	Descripción
Enlace Físico (<i>Hard Link</i>)	Un enlace físico permite que un archivo aparezca en más de un directorio. En lugar de una copia, es una nueva referencia al mismo contenido . Funcionamiento : El sistema de archivos utiliza un i-nodo para cada archivo; un enlace físico crea una referencia adicional a este i-nodo, apuntando al mismo contenido que el archivo original.
Enlace Simbólico (<i>Symbolic Link</i>)	Un enlace simbólico es un puntero a otro archivo o directorio, actuando como una referencia en lugar de un duplicado de contenido. Funcionamiento : No contiene el contenido ni el i-nodo del archivo original, solo la ruta al archivo o directorio enlazado . Si el archivo enlazado se mueve o elimina, el enlace simbólico queda roto e inválido.

Sistema de Archivos

En un sistema operativo, el **sistema de archivos** es la estructura que permite organizar y gestionar los archivos y directorios en el disco. Los discos físicos (como los discos duros o SSD) son donde se almacenan los sistemas de archivos, pero no siempre se utiliza todo el disco como una única unidad.

Distribución del sistema de archivos

4 particiones de disco, me dice que partición tengo activa. El MBR busca esa partición y la bootea. Cada partición tiene lo que se ve abajo



1. Los sistemas de archivos se almacenan en discos

Cada disco físico (o unidad de almacenamiento) tiene un **sistema de archivos** que organiza cómo se almacenan y acceden los archivos. El sistema de archivos define cómo se dividen los bloques de datos, cómo se gestionan los permisos y cómo se localizan los archivos en el disco. Ejemplos de sistemas de archivos son **NTFS** (en Windows), **ext4** (en Linux), y **FAT32** (en sistemas compatibles con múltiples dispositivos).

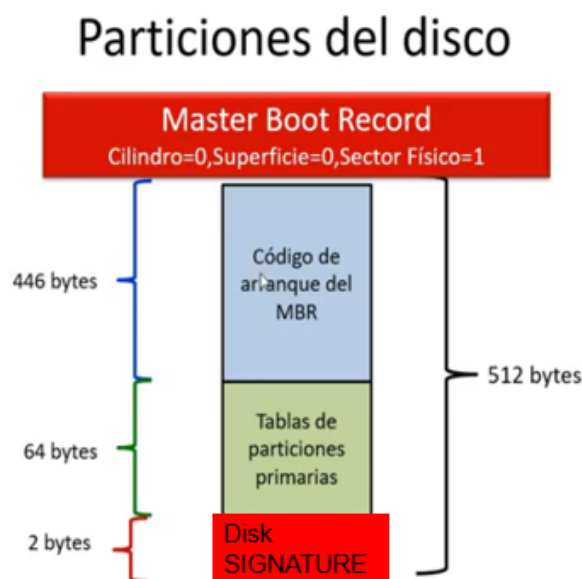
2. Particiones en el disco

Un disco físico puede dividirse en **varias particiones**, cada una de las cuales puede tener su propio sistema de archivos. Una partición es una división lógica del disco, y cada partición puede funcionar como si fuera un disco independiente. Esto permite instalar varios sistemas operativos en un solo disco o mantener diferentes sistemas de archivos en distintas secciones del mismo disco.

3. El sector 0: MBR

El **MBR** o **Master Boot Record** es un sector muy importante que se encuentra en el **sector 0** de un disco. Este sector es el primer lugar donde el sistema busca información cuando enciendes la computadora. El MBR contiene dos partes principales:

- **Código de arranque del MBR (Bootloader)**: ocupa los primeros 446 bytes y contiene el código necesario para iniciar el sistema operativo o cargar un gestor de arranque. Este código es lo primero que la computadora ejecuta al arrancar.
- **Tabla de particiones primarias**: ubicada en los siguientes 64 bytes, contiene información sobre hasta cuatro particiones primarias del disco. Cada entrada de la tabla de particiones ocupa 16 bytes y describe el tipo, tamaño y posición de cada partición en el disco.
- **Firma del disco**: en los últimos 2 bytes (0x55AA), indica datos macros del disco. Esta firma es necesaria para que el BIOS reconozca el MBR como válido y proceda con el proceso de arranque.



4. El superblock

El **Superblock** contiene todos los parámetros claves acerca del sistema de archivos y se lee en la memoria cuando se arranca la computadora o se entra en contacto con el sistema de archivos por primera vez.

Procesos de inicio de Arranque del sistema operativo

Cuando enciendes tu computadora, el **BIOS** (Basic Input/Output System) se activa primero. El BIOS es el firmware que inicializa el hardware del sistema y prepara el equipo para cargar el sistema operativo.

El proceso que sigue es el siguiente:

- **El BIOS lee el MBR**: El BIOS localiza el MBR en el sector 0 del disco de arranque. El MBR tiene las instrucciones básicas que el BIOS necesita para continuar el proceso de arranque.
- **El MBR localiza la partición activa**: El MBR contiene una tabla con todas las particiones del disco y, entre ellas, marca cuál es la **partición activa**. La partición activa es aquella que contiene el sistema operativo que se va a arrancar.
- **El MBR lee el bloque de arranque**: Una vez que se localiza la partición activa, el MBR lee el **bloque de arranque** o **boot sector** de esa partición. Este bloque contiene instrucciones específicas sobre cómo iniciar el sistema operativo dentro de esa partición.

Finalmente, el **bloque de arranque** ejecuta el código necesario para cargar el sistema operativo que está instalado en la partición.

El **sistema operativo (SO)** se carga en la memoria RAM y, una vez cargado, el control del sistema se transfiere a él, lo que te permite empezar a utilizar tu computadora.

Montaje

Permite montar un sistema de archivo en otro o una partición de disco en otra. Es decir, que, por ejemplo, un usb pueda estar disponible para su uso. En este proceso, el sistema asigna un punto de montaje donde se pueden acceder a los archivos y carpetas de esa partición. Por ejemplo con la instrucción:

```
mount("/dev/fd0", "/mnt", 0)
```

podemos montar en `/mnt` un archivo especial de bloque `/dev/fd0` para la unidad 0 y el tercer parámetro indica que el sistema de archivo se va a montar en modo de lectura-escritura o sólo de escritura.



Figura 1-22. (a) Sistema de archivos antes del montaje. (b) Sistema de archivos después del montaje.

El *daemon* es el encargado de avisarle al sistema de archivos que existe un pen-drive. El sistema de archivos lo monta automáticamente (automount), entonces el contenido de `mnt` desaparece y aparece el contenido del pen-drive (no desaparece, lo oculta).

Implementación

Asignación continua

En la **asignación continua** de archivos, se asigna al archivo una serie de bloques consecutivos en el disco para almacenar sus datos. Esto significa que si el archivo ocupa, por ejemplo, 10 bloques, esos 10 bloques estarán dispuestos uno detrás del otro, formando una secuencia continua en el disco.

En la siguiente figura 4-10 se puede ver la asignación contigua del espacio de disco para 7 archivos:

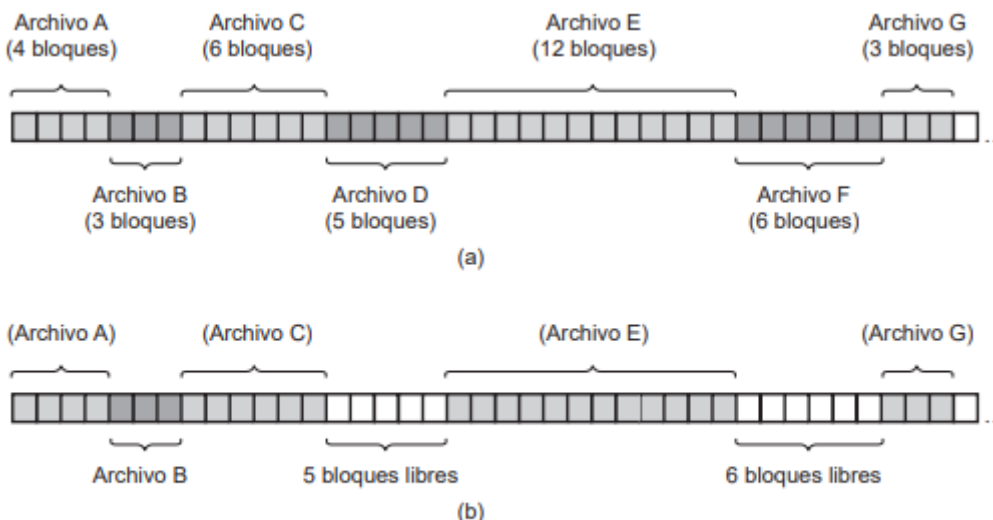


Figura 4-10. (a) Asignación contigua de espacio de disco para siete archivos. (b) El estado del disco después de haber removido los archivos D y F.

Características clave:

- **Inicio y longitud:** Para cada archivo, el sistema de archivos necesita conocer dos cosas: la **dirección de inicio** (el primer bloque donde comienza el archivo) y la **longitud** (el número de bloques que ocupa el archivo). Con esta información, el sistema puede acceder al archivo.
- **Acceso rápido:** Dado que los bloques están contiguos, el acceso a los datos es muy rápido. Una vez que el sistema encuentra el inicio del archivo, puede leer todos los bloques secuencialmente sin tener que buscar en diferentes partes del disco.

Ventajas

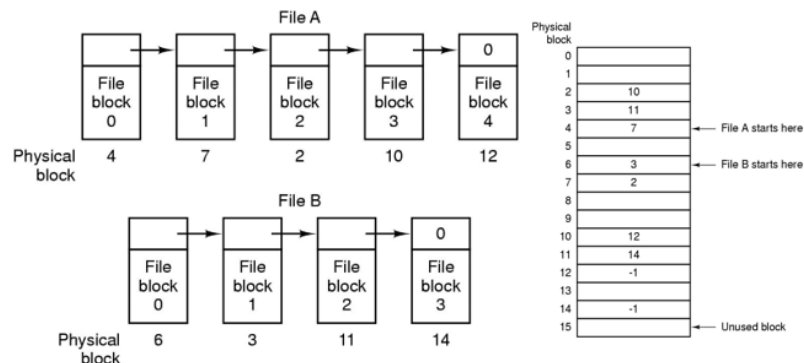
- Simple de implementar, se almacena cada archivo como una serie contigua de bloques de disco.
- Fácil de leer, solo es necesario una búsqueda.

Desventaja

- Fragmentación → porque tenemos bloques que no están ocupados al 100%.

Asignación de lista enlazada

En este esquema, cada archivo se representa como una **lista enlazada** de bloques en el disco. El primer bloque de un archivo contiene los datos iniciales y un puntero que señala al siguiente bloque donde continúa el archivo. El segundo bloque contiene más datos y un puntero al siguiente bloque, y así sucesivamente, hasta que se llega al último bloque del archivo, que contiene un puntero nulo (indicando que no hay más bloques).



Ventajas:

- Solo ocurre fragmentación interna en el último bloque de cada archivo.
- En la entrada del directorio es suficiente con almacenar la dirección de disco del primer bloque.

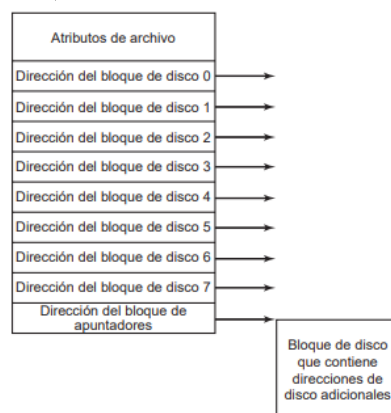
Desventajas:

- El acceso aleatorio es costoso.
- Agregar el puntero en la cabeza de cada bloque implica mayor sobrecarga durante la copia de archivos.
- Podemos tener la situación que se pierde un enlace y se rompe el archivo.

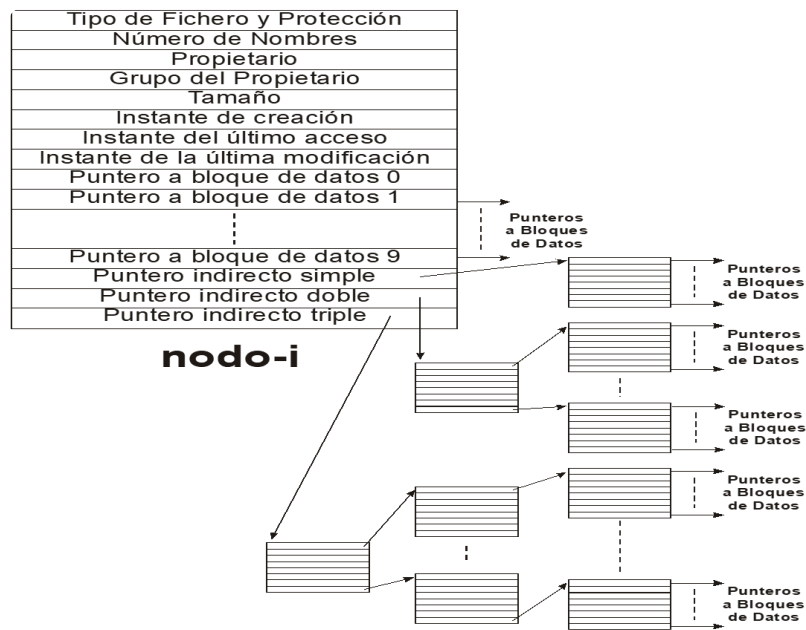
Para "arreglar" esta última desventaja podemos usar **listas doblemente enlazadas** como **FAT**

Index Node

Un **i-nodo** es una estructura que **contiene metadatos sobre un archivo o directorio**. Cada archivo y directorio en un sistema de archivos tiene su propio i-nodo. Los i-nodos almacenan la información necesaria para acceder a los bloques de datos del archivo en el disco, así como información adicional sobre el archivo.



La gran ventaja de este esquema, en comparación con los archivos vinculados que utilizan una tabla en memoria, es que el i-nodo necesita estar en memoria sólo cuando está abierto el archivo correspondiente. La desventaja es que cada i-nodo tiene un tamaño fijo.



El i-nodo **contiene varios atributos o metadatos** de un archivo o directorio, como:

1. **Tipo de archivo:** Si es un archivo regular, un directorio, un enlace simbólico, etc.
2. **Tamaño del archivo:** El tamaño total en bytes del archivo.
3. **Punteros a bloques de datos:** Direcciones de los bloques en disco donde se almacena el contenido del archivo.
4. **Permisos de acceso:** Indicando qué usuarios pueden leer, escribir o ejecutar el archivo.
5. **Propietario del archivo:** El identificador del usuario (UID) y grupo (GID) que posee el archivo.
6. **Fechas:** Tiempos de creación, modificación, y acceso del archivo.
7. **Número de enlaces:** El número de enlaces duros (hard links) que apuntan a este archivo.
8. **Información de control:** Como la referencia a las operaciones del sistema de archivos.

Estructura de los i-nodos

La implementación de los i-nodos está diseñada para permitir que los archivos crezcan y se gestionen de manera eficiente. Para lograr esto, los i-nodos utilizan una combinación de punteros directos e indirectos para referenciar los bloques de datos en el disco.

1. **Punteros directos:** Son punteros que apuntan directamente a bloques de datos en el disco. Si el archivo es pequeño, los punteros directos pueden ser suficientes para almacenar todos los datos del archivo.
2. **Puntero indirecto simple:** Si los punteros directos no son suficientes, el i-nodo tiene un puntero a un bloque de punteros. Este bloque de punteros contiene direcciones a más bloques de datos en el disco.
3. **Puntero indirecto doble:** Para archivos más grandes, el i-nodo incluye un puntero a un bloque que contiene punteros a bloques de punteros, y estos bloques de punteros a su vez contienen direcciones de bloques de datos.
4. **Puntero indirecto triple:** Si el archivo es aún más grande, se usa un puntero a un bloque que apunta a bloques de punteros, que a su vez contienen punteros a bloques de punteros, los cuales finalmente apuntan a los bloques de datos.

$$\text{Tamaño máximo del archivo: } 10Tb + \frac{Tb}{4}Tb + \left(\frac{Tb}{4}\right)^2Tb + \left(\frac{Tb}{4}\right)^3Tb$$

Tb : tamaño del bloque y direcciones de bloques de 4 bytes.

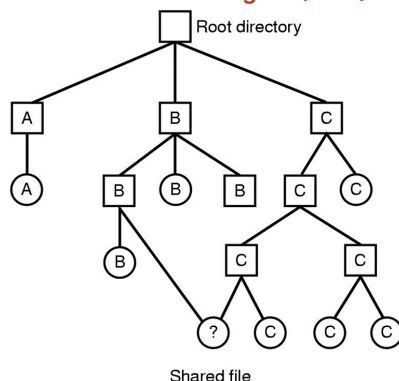
Archivos compartidos

Se refiere a la posibilidad de que varios usuarios puedan acceder y utilizar el mismo archivo, lo que lleva a una estructura más compleja de organización del sistema de archivos.

La conexión o referencia entre dos directorios que comparten el acceso a un archivo se conoce como un **vínculo o enlace**. Esto permite que un archivo sea referenciado desde varios directorios sin necesidad de duplicarlo.

Existen dos tipos principales de enlaces: [HardLink](#) o [SoftLink](#)

Cuando los archivos se comparten entre múltiples directorios, la estructura del sistema de archivos deja de ser un árbol, que es una estructura jerárquica simple en la que cada archivo o directorio tiene un solo padre. En su lugar, el sistema de archivos se convierte en un **Grafo Acíclico Dirigido (DAG)**.



Ocurren ciertos problemas al compartir archivos como que si el archivo está siendo accedido por dos usuarios y uno modifica el archivo, los cambios no se verían reflejados para el otro usuario, por lo que hay dos formas de **solucionar** este problema. En la primera solución, los bloques de disco no se listan en los directorios, sino en una pequeña estructura de datos asociada con el archivo en sí. Entonces, los directorios apuntarían sólo a la pequeña estructura de datos (la pequeña estructura de datos es el i-nodo).

La segunda solución, B se vincula a uno de los archivos de C haciendo que el sistema cree un archivo, de tipo LINK e introduciendo ese archivo en el directorio de B. El nuevo archivo contiene sólo el nombre de la ruta del archivo al cual está vinculado. Cuando B lee del archivo vinculado, el sistema operativo ve que el archivo del que se están leyendo datos es de tipo LINK, busca el nombre del archivo y lee el archivo. A este esquema se le conoce como **vínculo simbólico (liga simbólica)**.

Cada uno de estos métodos tiene su desventaja. El primero queda en sub-propiedad de B, aunque el archivo pertenezca a C. En el segundo método el archivo se destruye y B no puede acceder a este.

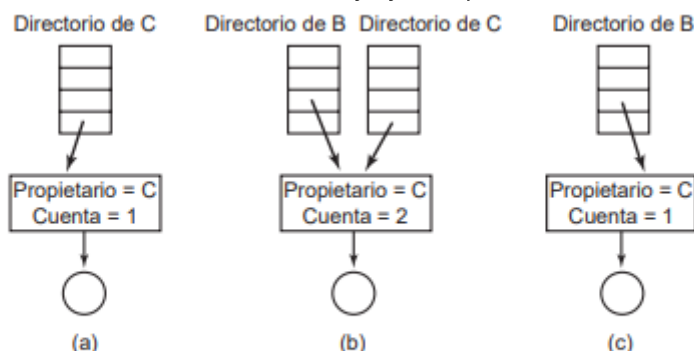


Figura 4-17. (a) Situación previa a la vinculación. (b) Después de crear el vínculo. (c) Después de que el propietario original elimina el archivo.

VFS (Virtual File System)

El VFS actúa como una **capa de abstracción** que encapsula las diferencias entre los distintos sistemas de archivos. Esto significa que no importa si el sistema de archivos subyacente es ext4, NTFS, FAT, o cualquier otro; desde el punto de vista de las aplicaciones o del propio sistema operativo, todas las operaciones sobre archivos (leer, escribir, crear, eliminar) se manejan de la misma forma.

El VFS permite que el acceso a los archivos sea **independiente del sistema de archivos subyacente**. Ya sea que los archivos estén almacenados en un sistema de archivos ext4, FAT, o cualquier otro, las operaciones sobre esos archivos son homogéneas. Esto significa que los comandos que usamos para manipular archivos funcionan igual sin importar el tipo de sistema de archivos.

Utilización del Magic Number

El **VFS necesita “conocer” estos magic numbers** para poder identificar correctamente qué tipo de sistema de archivos está montado. Cuando un sistema operativo monta un dispositivo de almacenamiento, el VFS lee este número mágico para determinar a qué tipo de sistema de archivos pertenece y luego interactúa con él de la manera adecuada.

Administración del espacio libre en la memoria

Un disco duro o dispositivo de almacenamiento se divide en unidades llamadas **bloques**, que son las unidades mínimas de almacenamiento que el sistema operativo utiliza para leer y escribir datos.

¿Qué es el tamaño de bloque?

- El **tamaño de bloque** se refiere a la cantidad de datos que un bloque individual puede almacenar. Este tamaño es fijo y puede variar dependiendo del sistema de archivos, pero comúnmente es de 4KB, aunque puede ser más grande o más pequeño según las configuraciones.

Registro de bloques libres en disco

El **registro de bloques libres** es un mecanismo que utiliza el sistema operativo para llevar un control de los bloques que están disponibles o libres en el disco, es decir, aquellos que no están ocupados por ningún archivo.

- Asignación de espacio:** Cuando se necesita guardar un archivo nuevo, el sistema operativo debe saber qué bloques están disponibles para asignárselos al archivo. Mantener un registro de bloques libres permite asignar espacio de manera rápida y eficiente.
 - Liberación de espacio:** Cuando se elimina un archivo, los bloques que estaban asignados a ese archivo deben ser marcados como libres para que puedan ser reutilizados.
- Hay varios métodos que los sistemas de archivos utilizan para llevar un registro de los bloques libres:

1. Mapa de bits (*bitmap*):

- En este esquema, el sistema operativo utiliza un mapa de bits en el que cada bit representa un bloque del disco. Un bit de valor 1 indica que el bloque está ocupado, mientras que un bit de valor 0 indica que el bloque está libre.
- Ventajas:** Es eficiente en términos de espacio porque cada bloque solo requiere un bit en el mapa, lo que significa que se puede registrar el estado de muchos bloques con una cantidad relativamente pequeña de memoria.
- Desventajas:** Si el disco es muy grande, el mapa de bits puede volverse complicado de gestionar.

2. Lista enlazada:

- En este método, los bloques libres se encadenan en una lista enlazada. Cada bloque libre contiene un puntero al siguiente bloque libre. Cuando el sistema operativo necesita un bloque, toma el primero de la lista.
- Ventajas:** Es fácil de implementar y no requiere mucho espacio extra.
- Desventajas:** Puede ser menos eficiente a la hora de buscar bloques contiguos para archivos grandes, ya que implica seguir los enlaces entre bloques.

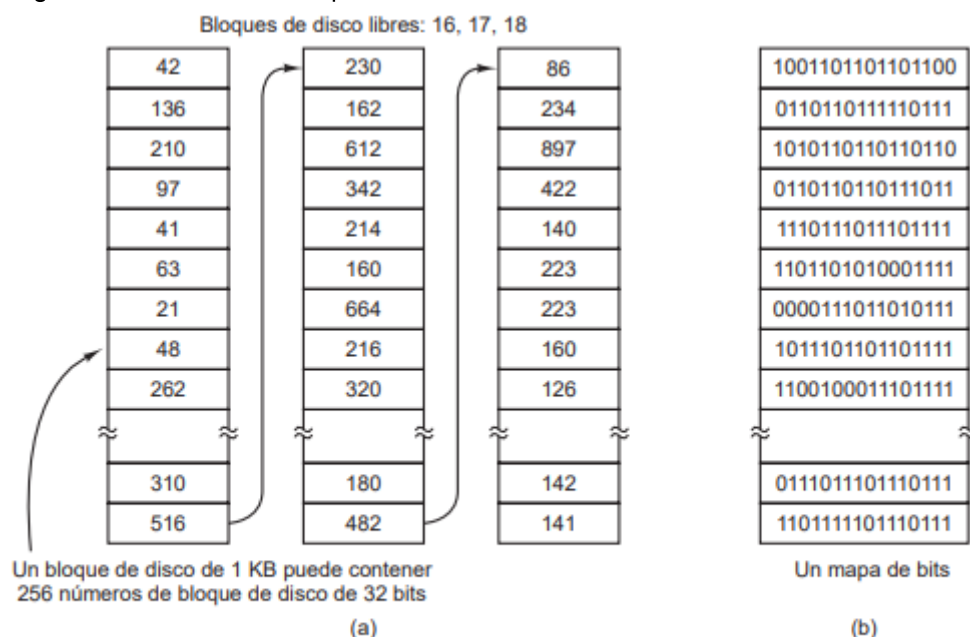
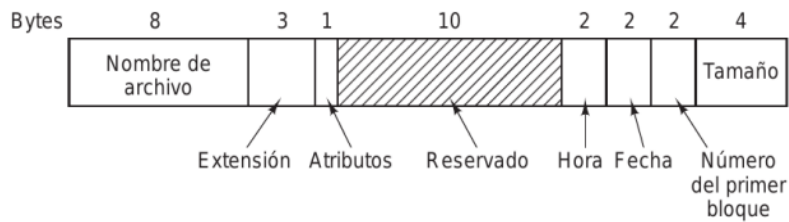


Figura 4-22. (a) Almacenamiento de la lista de bloques libres en una lista enlazada.
(b) Un mapa de bits.

MS-DOS (Microsoft Disk Operating System) es un sistema operativo que utiliza un sistema de archivos conocido como **FAT** para llevar un registro de cómo están organizados los archivos en el disco. En el sistema FAT, **cada archivo tiene una entrada en el directorio**. Esta entrada contiene información sobre el archivo, como su nombre, tamaño y, lo más importante, el número del **primer bloque** donde comienza el archivo.

Estructura de la Entrada de Directorio

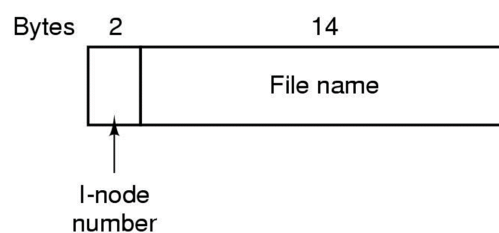
Una entrada de directorio típica en MS-DOS consta de varios campos que almacenan información específica. Los componentes principales son:



- **Nombre del Archivo:**
 - Contiene el nombre del archivo, generalmente limitado a **8 caracteres** para el nombre y **3 caracteres** para la extensión (ejemplo: **ARCHIVO.TXT**).
- **Atributos:**
 - Un conjunto de bits que define propiedades del archivo, como si es de solo lectura, oculto, un directorio, o un archivo del sistema.
 - Los atributos se almacenan en un campo específico, y cada bit representa una característica diferente.
- **Tamaño del Archivo:**
 - Un campo que indica el tamaño total del archivo en bytes. Esto permite al sistema saber cuánto espacio ocupará el archivo en el disco.
- **Fecha y Hora de Creación:**
 - Registra la fecha y la hora en que se creó el archivo o se modificó por última vez. Este dato es útil para la gestión y organización de archivos.
- **Número del Primer Bloque:**
 - Este campo almacena el número del primer bloque o cluster donde se encuentra el archivo en el disco. Este es el dato más crítico, ya que permite al sistema localizar rápidamente los datos del archivo utilizando la Tabla de Asignación de Archivos (FAT).
- **Enlaces:**
 - Indica cuántas veces se ha enlazado este archivo en diferentes directorios. Esto es útil para determinar la cantidad de referencias que tiene un archivo.

V7 UNIX File System

El **sistema de archivos** está en la forma de un árbol que empieza en el directorio raíz, con la adición de vínculos para formar un gráfico acíclico dirigido. Los nombres de archivos tienen hasta 14 caracteres y pueden contener cualquier carácter ASCII excepto / (debido a que es el separador entre los componentes en una ruta) y NUL (debido a que se utiliza para rellenar los nombres menores de 14 caracteres). NUL tiene el valor numérico de 0. Una entrada de directorio de UNIX contiene una entrada para cada archivo en ese directorio. Cada entrada es en extremo simple, ya que UNIX utiliza el esquema de nodos-i. Una entrada de directorio sólo contiene dos campos: el nombre del archivo (14 bytes) y el número del nodo-i para ese archivo (2 bytes). Estos parámetros limitan el número de archivos por cada sistema de archivos a 64 K.



Para poder manejar archivos muy grandes. Las primeras 10 direcciones de disco se almacenan en el mismo nodo-i, por lo que para los archivos pequeños toda la información necesaria se encuentra justo en el nodo-i, que se obtiene del disco a la memoria principal al momento de abrir el archivo. Para archivos un poco más grandes,

una de las direcciones en el nodo-i es la dirección de un bloque de disco llamado **bloque indirecto sencillo**. Este bloque contiene direcciones de disco adicionales. Si esto no es suficiente, hay otra dirección en el nodo-i, llamada **bloque indirecto doble**, que contiene la dirección de un bloque que contiene una lista de bloques indirectos sencillos. Cada uno de estos bloques indirectos sencillos apunta a unos cuantos cientos de bloques de datos. Si esto aún no es suficiente, también se puede utilizar un **bloque indirecto triple**

Ejemplo de como buscaría UNIX V7 un archivo

Consideremos cómo se busca el nombre de la ruta `/usr/ast/mbox`.

1. En primer lugar, el sistema de archivos **localiza el directorio raíz**. En UNIX, su nodo-i se localiza en un lugar fijo en el disco. De este nodo-i localiza el directorio raíz, que puede estar en cualquier parte del disco, pero digamos que está en el bloque 1.
2. Después **lee el directorio raíz y busca el primer componente de la ruta**, `usr`, en el directorio raíz para encontrar el número de nodo-i del archivo `/usr`. La acción de localizar un nodo-i a partir de su número es simple, ya que cada uno tiene una ubicación fija en el disco.
3. A partir de este nodo-i, el sistema **localiza el directorio** para `/usr` y busca el siguiente componente, `ast`, en él. Cuando encuentra la entrada para `ast`, tiene el nodo-i para el directorio `/usr/ast`.
4. A partir de este nodo-i puede **buscar** `mbox` en el mismo directorio.
5. Después, el nodo-i para este archivo se **lee en memoria y se mantiene** ahí hasta que se cierra el archivo.

